

A Memory-Free Number-Theoretic Classifier for π -Membership of Hexadecimal Sequences

@DepartureNo2452

February 2026; revised September 2026 (v2)

Revision note

Version 1 of this paper was titled *A Number-Theoretic Classifier via Inverse Application of the Bailey–Borwein–Plouffe Formula*. This version corrects three substantive errors and retires two overclaims. In brief:

1. **The classification threshold was not tied to the sequence resolution.** With $\varepsilon = 10^{-5}$ fixed and $d = 7$ hex digits, the classifier certified only about four digits of agreement, and the true false-positive rate was 1.19×10^{-2} rather than the claimed 2.2×10^{-6} : a factor of roughly 5300. Section 2.5 now derives the correct rate, and the single experimental failure in v1 disappears once the threshold is set properly.
2. **The $\sim 10^{-9}$ residual at matching positions was misattributed to float64 rounding.** It is truncation error from representing a d -digit prefix, and is larger than the engine’s actual numerical error by about five orders of magnitude. This matters because it, not rounding, is what sets the achievable threshold.
3. **The false-positive table contained an arithmetic error:** three rows omitted the factor W .
4. The word “inverse” has been removed from the title and method description. The procedure evaluates BBP *forward* at every candidate position; it is a bounded positional search, not an inversion.
5. The claimed analogy to one-way functions has been reduced to a short remark, since the inverse problem admits an obvious practical shortcut that v1 did not consider.

Additions: a measured error budget (Section 2.5), a corrected and implemented disjoint-block method for long sequences (Section 2.7), a strided sweep giving a measured $3.9\times$ speedup (Section 2.8), and Monte Carlo validation of the false-positive law over 200,000 trials (Section 5.7). A full errata list is in Appendix C.

Abstract

We present a binary classifier that determines whether a given hexadecimal digit sequence belongs to the expansion of π , and if so at what position, by exploiting the positional independence of the Bailey–Borwein–Plouffe (BBP) formula. The BBP formula permits extraction of the n th hexadecimal digit of π without computing any preceding digit. Given an unknown hex sequence and a bounded positional neighborhood, we sweep candidate positions, evaluate BBP at each, and test

for congruence on a modulo-1 circle. The classifier requires no precomputed digit tables and $O(1)$ memory regardless of position.

Its error behaviour is governed by a single quantity, the *effective resolution* $\rho = \max(16^{-d}, \delta(n))$, where d is the sequence length in hex digits and $\delta(n)$ is the numerical error of the BBP engine. False negatives are impossible provided the acceptance threshold satisfies $\varepsilon > \rho$; the false-positive rate is then $\approx 2W\varepsilon$ for a window of W positions, minimised at $\varepsilon \approx \rho$. Since $\delta(n)$ grows like $\sqrt{n}$ and reaches 7×10^{-14} by $n = 20,000$, IEEE 754 double precision resolves at most $d_{\max} \approx 11$ digits in a single evaluation; beyond that, partitioning the sequence into disjoint blocks and requiring simultaneous resonance restores exponential decay in d at a cost of 2^B for B blocks. We verify each of these laws empirically, including a 200,000-trial Monte Carlo estimate of the false-positive rate agreeing with prediction to within 3%, and validation of 21-digit classification at predicted rate 2.5×10^{-22} .

1. Introduction

The Bailey–Borwein–Plouffe formula, discovered by Plouffe in 1995 and published by Bailey, Borwein and Plouffe in 1997, is

$$\pi = \sum_{k=0}^{\infty} \frac{1}{16^k} \left(\frac{4}{8k+1} - \frac{2}{8k+4} - \frac{1}{8k+5} - \frac{1}{8k+6} \right).$$

Its remarkable property is that it permits computation of hexadecimal digits of π starting at an arbitrary position n without computing the preceding $n-1$ digits. This is accomplished through modular exponentiation: the operation `pow(16, n-k, 8k+j)` discards all information about prior digits, leaving only what is needed to recover the fractional part of $16^n\pi$.

The formula is normally used in the forward direction: given n , produce the digits at n . Applications include distributed verification of large π computations (PiHex; Percival, 2000) and digit extraction at positions exceeding 10^{17} (Karrels, 2017).

We consider the converse question. Given a hex sequence of unknown provenance and a bounded estimate of where in π it might lie, does it occur there, and at what position? This is not an analytic inversion, and we no longer describe it as one: the modular exponentiation at the heart of BBP destroys any continuous structure in n , so there is no gradient to descend and no algebraic route from digits back to position. What remains is search. The contribution here is not the search itself, which is elementary, but the observation that BBP makes it possible with $O(1)$ memory and no digit table, together with a correct account of the resulting error rates.

The scalar object we sweep over has appeared before. Bailey (2006) notes that a single BBP evaluation yields not one digit but the first several hex digits following the target position. More directly, the mapping $n \mapsto \text{frac}(16^n\pi)$ has been studied as the “ π fraction” by Sobieszczanski-Sobieski (2014), who used its empirical uniformity on $[0, 1]$ – the first 215,829 values having mean ≈ 0.4993 – to generate sampling points for global optimisation. That empirical uniformity is precisely the assumption our false-positive analysis requires, and we cite it in place of appealing only to the unproven base-16 normality of π .

2. Method

2.1 Forward BBP engine

For parameter $j \in \{1, 4, 5, 6\}$ and target position n , define

$$S_j(n) = \sum_{k=0}^n \frac{16^{n-k} \bmod (8k+j)}{8k+j} + \sum_{k=n+1}^{\infty} \frac{16^{n-k}}{8k+j}.$$

The first, finite sum uses modular exponentiation to prevent overflow: $16^{n-k} \bmod (8k+j)$ is computed in $O(\log n)$ time by repeated squaring. The second sum converges geometrically and is truncated after 20 terms, contributing an error below $16^{-20} \approx 8.6 \times 10^{-25}$. The fractional part of $16^n \pi$ is then

$$f(n) = \text{frac}(4S_1(n) - 2S_4(n) - S_5(n) - S_6(n)) \in [0, 1).$$

The key structural point is that $f(n)$ is a *scalar*, not a string. Hex digits are recovered by repeatedly multiplying by 16 and taking integer parts, but the classifier never needs to do so: it operates on the scalar directly.

2.2 Input encoding

A d -digit hexadecimal sequence maps to a target scalar $t \in [0, 1)$:

$$t = \sum_{i=1}^d h_i 16^{-i},$$

where h_i is the value of the i th digit. For example 243F maps to $2 \cdot 16^{-1} + 4 \cdot 16^{-2} + 3 \cdot 16^{-3} + 15 \cdot 16^{-4} \approx 0.14184$.

Note that this encoding is *lossy in a specific and important way*: t determines the first d digits and says nothing about the rest. A true match therefore does not give $f(n) = t$ but $f(n) = t + \tau$, where $\tau \in [0, 16^{-d})$ is the value of the digits beyond the d th. This residual is the dominant term in the error budget and was misidentified in v1.

2.3 Congruence loss on the modulo-1 circle

Since $f(n)$ and t both lie in $[0, 1)$, their distance is measured on a circle of circumference 1:

$$L(n, t) = \min(|f(n) - t|, 1 - |f(n) - t|).$$

The circular metric handles wrap-around correctly, so that values near 0.999 and 0.001 are recognised as close. This matters when the target begins with a run of Fs or 0s and the residual crosses the boundary.

2.4 Classification by resonance sweep

Given a sequence and a window $[n_{lo}, n_{hi})$ of size W , the classifier evaluates $L(n, t)$ at each integer n in the window. If any n yields $L < \varepsilon$, the sequence is reported as occurring in π at position n ; otherwise it is reported as not occurring within the searched range. The whole content of the method is now in the choice of ε .

2.5 The error budget

Three distinct quantities limit how finely the classifier can discriminate. Conflating them was the principal error of v1.

Truncation, 16^{-d} . As shown in Section 2.2, a genuine match leaves a residual $\tau \in [0, 16^{-d})$. Under the uniformity of π 's digits, τ is approximately uniform on that interval, with mean $16^{-d}/2$. For $d = 7$ this gives a floor of 3.73×10^{-9} and a typical residual of 1.9×10^{-9} .

Numerical error, $\delta(n)$. The finite sum accumulates n floating-point roundings, each of relative size $\sim 2^{-53}$, in a random-walk fashion. Measuring against π computed to 30,000 decimal digits (Section 5.1) gives $\delta(n) \approx C\sqrt{n}$ with $C \lesssim 5.5 \times 10^{-16}$ observed; we adopt the conservative $C = 10^{-15}$. Thus $\delta(600) \approx 2.5 \times 10^{-14}$ and $\delta(2 \times 10^4) \approx 1.4 \times 10^{-13}$.

Threshold, ε . Chosen by the user.

Define the **effective resolution** $\rho(d, n) = 16^{-d} + \delta(n)$. Then:

- *Zero false negatives* requires $\varepsilon > \rho(d, n_{hi})$. Otherwise a true match can be rejected because its truncation residual exceeds the threshold.
- *False positives* occur when some non-matching $f(n)$ lands within ε of t . Treating the W values $f(n)$ as uniform and approximately independent, each contributes probability 2ε (the arc length of the acceptance ball), giving

$$P(\text{false positive}) \approx 1 - (1 - 2\varepsilon)^W \approx 2W\varepsilon.$$

These pull in opposite directions through a single knob, so the optimum is to take ε as small as the false-negative constraint permits:

$$\varepsilon^* = (1 + \gamma) \rho(d, n_{hi}), \quad P(\text{FP}) \approx 2W\rho(d, n_{hi}),$$

with a small safety margin γ ($\gamma = 1/4$ throughout this paper, covering slack in the fit for δ).

Two regimes follow. When $16^{-d} \gg \delta(n)$ the classifier is *truncation-limited* and $P(\text{FP}) \approx 2W \cdot 16^{-d}$, decaying exponentially in d as v1 claimed (up to the factor 2 that v1 omitted). When $16^{-d} \lesssim \delta(n)$ it is *precision-limited*, $P(\text{FP}) \approx 2W\delta(n)$, and **adding digits buys nothing**. The crossover is at

$$d_{\max}(n) = \left\lfloor \log_{16} \frac{1}{\delta(n)} \right\rfloor,$$

giving $d_{\max} = 11$ for $n \leq 10^3$, 10 at $n = 10^4$, 9 at 10^6 , 8 at 10^9 , and 7 at 10^{12} . Single-evaluation classification of the 20- and 40-digit sequences tabulated in v1 is therefore not possible in double precision at any position.

Corrected table, replacing the one in v1 §2.5 (whose last three rows were additionally off by a factor $W = 1000$):

d	W	v1 claim	corrected, single evaluation	feasible in float64?	disjoint blocks ($b = 7$)
7	100	3.7×10^{-7}	9.3×10^{-7}	yes	7.5×10^{-7}
7	600	2.2×10^{-6}	5.6×10^{-6}	yes	4.5×10^{-6}
10	1000	9.1×10^{-13}	2.4×10^{-9}	yes	3.6×10^{-9}
20	1000	8.3×10^{-25}	7.9×10^{-11}	no	6.6×10^{-21}
40	1000	7.2×10^{-49}	7.9×10^{-11}	no	4.4×10^{-44}

The v1 figures for $d = 20$ and $d = 40$ are unreachable by any method described here. The last column shows what the block method of Section 2.7 actually delivers, which is still strong: combinatorial impossibility in practice, just not by 24 orders of magnitude more than the arithmetic allows.

2.6 Choosing the threshold in practice

The threshold should never be a fixed constant. The implementation sets

$$\varepsilon = \frac{5}{4} (16^{-d} + 10^{-15} \sqrt{n_{hi}}),$$

computed from the sequence length and window actually supplied. For $d = 7$, $W = 600$ this gives $\varepsilon = 4.66 \times 10^{-9}$, versus the 10^{-5} used in v1 – a ball 2100 times narrower, and the reason the v1 false positive vanishes.

2.7 Disjoint blocks for long sequences

For $d > d_{\max}(n)$ a single evaluation cannot resolve the whole sequence. Partition the d digits into B **disjoint** blocks of sizes $b_1, \dots, b_B$ with $\sum b_i = d$, each $b_i \leq d_{\max}(n)$. Block i begins at offset $o_i = \sum_{l < i} b_l$, and the classifier requires

$$L(n + o_i, t_i) < \varepsilon_i \quad \text{for every } i,$$

with ε_i set from b_i by the rule above. Disjointness is what makes the per-block events independent, so the probabilities multiply:

$$P(\text{FP}) \approx W \prod_{i=1}^B 2\varepsilon_i \approx W 2^B 16^{-d}.$$

This recovers the full exponential decay in d , at a cost of 2^B relative to an idealised single check. For $d = 21$, $b = 7$, $W = 600$: 2.5×10^{-22} .

V1 §2.6 proposed *overlapping* windows (digits 0–7 at n , digits 5–12 at $n + 5$) and claimed the conjunction gives “the confidence equivalent of the full sequence length.” It does not. Overlapping

blocks share digits, so the shared digits are tested twice and contribute no independent information: the example given covers digits 0–12, yielding 13 digits of discrimination rather than 16. Disjoint blocks are required for the product law. V1 also never implemented or tested the technique; Sections 5.8 and 5.9 do.

2.8 Strided sweep

One BBP evaluation determines $f(n)$ and hence, by $f(n+i) = \text{frac}(16^i f(n))$, the values at the next several positions. Positions $n, \dots, n+s-1$ can therefore be tested from a single evaluation. Each shift multiplies the numerical error by 16, so the stride is bounded by requiring the amplified error to stay well below the resolution:

$$16^{s-1} \delta(n) < \alpha \cdot 16^{-d}, \quad \alpha = 0.1.$$

For $d = 7$ this gives $s = 4$ at $n \leq 10^3$, $s = 3$ at $n = 10^4$ – 10^6 , and $s = 1$ by $n = 10^9$, where float64 has no headroom left. The measured speedup at $W = 600$, $n \leq 600$ is $3.9\times$ with identical classifications (Section 5.10). The sweep cost falls to $O((W/s) n \log n)$.

3. Structure of the loss landscape

The congruence loss cannot be minimised by gradient descent or any continuous method. Because $f(n)$ depends on n through modular exponentiation, adjacent integer positions produce unrelated values. Measured around a known position $n = 42$:

n	37	38	39	40	41	42	43	44	45	46	47
L	.114	.488	.487	.459	.020	≈ 0	.328	.427	.491	.191	.380

There is no gradient signal: the loss at $n = 41$ says nothing about the solution at $n = 42$. The landscape is white noise with a single spike.

This is worth stating because it rules out optimisation as a search strategy, which is a real constraint on implementations. V1 went further and argued the structure is “operationally analogous to a one-way function.” We withdraw that framing, for three reasons.

First, the chaotic landscape is not an independent finding: it is a restatement of the same digit-uniformity assumption the false-positive analysis already rests on. Second, there is no secret and no trapdoor; the search space is bounded by the user’s positional prior rather than by any hardness, so “hard to invert” is doing no work. Third, and most concretely, the claim that inversion “admits no shortcut beyond exhaustive search” is false in the range that matters. The hexadecimal expansion of π has been computed and published to far more digits than this classifier can practically reach, and an indexed lookup in a stored expansion beats a BBP sweep by orders of magnitude for any position in that range. The asymmetry between forward and reverse evaluation here is real but ordinary, and carries no cryptographic content.

4. Computational complexity

A single BBP evaluation at position n performs n modular exponentiations of $O(\log n)$ multiplications each, so $O(n \log n)$ arithmetic operations, with $O(1)$ memory. The naive sweep over a window of size W costs $O(Wn \log n)$; the strided sweep of Section 2.8 costs $O((W/s)n \log n)$.

Measured (Python 3.12, single core, $3.1 \mu\text{s}$ per inner term), window $W = 100$:

position n	naive sweep	strided sweep	stride s
10^2	0.005 s	0.001 s	4
10^3	0.092 s	0.023 s	4
10^4	1.24 s	0.42 s	3
10^6	≈ 5 min (extrapolated)	≈ 2 min	3
10^9	≈ 86 h (extrapolated)	≈ 86 h	1

V1’s timing table was pessimistic by roughly an order of magnitude at moderate n and optimistic at the top end: a C implementation at a generous $50\times$ speedup puts $n = 10^9$, $W = 100$ at one to two hours, not minutes. Note also that at $n = 10^9$ the stride collapses to 1 and $d_{\max} = 8$, so the largest positions are exactly where the precision budget is tightest.

The advantage over stored-digit search is memory, not time: a handful of accumulators versus a digit table proportional to n . This is decisive only where no table exists, which means positions beyond published expansions. For positions within them, indexed lookup is faster and the honest recommendation is to use it.

5. Experimental results

All results below were produced by the implementation in Appendix A. Positive controls use sequences extracted from π at known positions; negatives use the hexadecimal expansion of e and pseudorandom strings.

5.1 Engine accuracy against ground truth

Comparing `bbp_fraction(n)` against `frac( $16^n\pi$ )` computed at 30,000 decimal digits:

n	10^2	2×10^2	10^3	2×10^3	5×10^3	10^4	2×10^4
$\delta(n)$	8.9e−16	3.2e−15	1.6e−14	2.4e−14	2.1e−15	1.2e−14	7.1e−14

The error is noisy, as expected for a random walk, with $\max_n \delta(n)/\sqrt{n} = 5.4 \times 10^{-16}$ over the sampled range. This justifies $C = 10^{-15}$ as a conservative bound and gives the $d_{\max}$ values of Section 2.5.

5.2 The loss floor is truncation, not rounding

For each position, the observed loss at the correct position is compared against the predicted truncation residual $\tau = f(n+7) \cdot 16^{-7}$ and against the measured numerical error:

n	sequence	observed L	predicted τ	$\delta(n)$
0	243F6A8	1.9842e−9	1.9842e−9	1.0e−16
10	A308D31	7.2172e−10	7.2172e−10	2.5e−16
42	9F31D00	1.9054e−9	1.9054e−9	5.4e−16
100	29B7C97	2.8675e−9	2.8675e−9	8.9e−16
500	86AF7C7	6.7809e−10	6.7810e−10	3.6e−15
1000	49F1C09	2.5678e−9	2.5678e−9	1.6e−14
2000	8326037	1.4866e−9	1.4866e−9	2.4e−14

Observation matches the truncation prediction to every digit printed, while the engine’s actual error is five orders of magnitude smaller. Every observed loss lies in $[0, 16^{-7})$, which is the signature of truncation. V1 reported the same values and attributed them to float64 rounding, and further claimed they grow with n for that reason; the table shows $n = 2000$ producing a *smaller* loss than $n = 100$. The variation is simply which digits happen to follow the prefix.

5.3 Positive tests ($d = 7$, ε tied to resolution)

n	sequence	window	loss	ε	result
0	243F6A8	[0, 51)	1.98e−9	4.66e−9	PASS
1	43F6A88	[0, 52)	1.94e−9	4.66e−9	PASS
10	A308D31	[0, 61)	7.22e−10	4.66e−9	PASS
42	9F31D00	[0, 93)	1.91e−9	4.66e−9	PASS
100	29B7C97	[50, 151)	2.87e−9	4.66e−9	PASS
200	F12C7F9	[150, 251)	2.13e−9	4.66e−9	PASS
350	F0CA417	[300, 401)	2.12e−9	4.66e−9	PASS
500	86AF7C7	[450, 551)	6.78e−10	4.66e−9	PASS

All identified at their exact positions. The margin between the largest observed loss and ε is narrow by design: that narrowness is what buys the false-positive rate.

5.4 Wide-search positive tests ($W = 600$)

n	sequence	loss	result
7	885A308	3.07e−9	PASS
75	01377BE	1.23e−9	PASS
250	574E69A	1.01e−9	PASS

5.5 Negative tests: Euler’s number

Slices of e in hexadecimal, swept over $[0, 600)$ with $\varepsilon = 4.66 \times 10^{-9}$:

slice	sequence	best loss	result
[0:7]	B7E1516	3.64e−4	PASS (no resonance)

slice	sequence	best loss	result
[10:17]	ED2A6AB	9.73e−4	PASS
[30:37]	C762E71	7.98e−4	PASS
[50:57]	84D9045	5.05e−5	PASS
[80:87]	F4BF8D8	6.76e−4	PASS

All rejected by margins of four to five orders of magnitude. Note that 84D9045 came within a factor of 5 of v1’s fixed $\varepsilon = 10^{-5}$: under the old threshold this test was closer to failing than v1’s presentation suggested.

5.6 Negative tests: pseudorandom noise

Same five sequences as v1 (seed 2026), swept over $[0, 600)$:

sequence	best loss	at n	v1 ($\varepsilon = 10^{-5}$)	v2 ($\varepsilon = 4.66\text{e-}9$)
3A37DFE	4.86e−4	378	reject	reject
702393E	1.22e−3	26	reject	reject
OFA6C8B	4.15e−3	476	reject	reject
BC2A299	1.25e−7	524	false positive	reject
E490BBE	2.77e−3	81	reject	reject

The single failure reported in v1 is a threshold artefact. BC2A299 agrees with π near position 524 to about 5.7 hex digits, which clears a ball of radius 10^{-5} but not one of radius 16^{-7} . Correcting the threshold turns v1’s 20/21 into 21/21.

V1 defended this false positive as “consistent with the predicted rate of $\approx 2.2 \times 10^{-6}$ – over 21 trials, one collision at this rate is not unexpected.” It is not: the expected count is 4.6×10^{-5} , so observing one is roughly a 1-in-20,000 event. Under the corrected rate of 1.2×10^{-2} the expected count is 0.25 and one collision is unremarkable. The data was already evidence against the v1 model.

5.7 Monte Carlo validation of the false-positive law

200,000 uniform random d -digit targets swept over $W = 600$ cached positions:

d	ε^*	observed FP rate	predicted $2W\varepsilon$	at fixed $\varepsilon = 10^{-5}$	v1 claim $W/16^d$
4	1.91e−5	2.23e−2	2.29e−2	1.21e−2	9.2e−3
5	1.19e−6	1.36e−3	1.43e−3	1.19e−2	5.7e−4
7	4.66e−9	1.50e−5	5.59e−6	1.19e−2	2.2e−6

The $2W\varepsilon$ law is confirmed to within 3% at $d = 4$ and 5% at $d = 5$. At $d = 7$ the observed rate rests on 3 hits against 1.1 expected, consistent under Poisson statistics ($p \approx 0.10$) and the largest deviation available at that sample size.

The fourth column is the decisive one. With ε held at 10^{-5} the false-positive rate is 1.2×10^{-2} *independent of d* : it does not improve at all between 4 and 7 digits, because the threshold and

not the sequence length is what determines discrimination. V1’s central claim that the error rate decays exponentially in sequence length is false for the classifier as v1 implemented it, and true only once the threshold is tied to d .

5.8 The product law for disjoint blocks

Two disjoint blocks of $b = 3$ digits ($B = 2$, $d = 6$), $W = 600$, 200,000 trials:

quantity	value
observed FP rate	2.05e−4 (41 hits)
predicted $W(2\varepsilon)^2$	2.24e−4
idealised single 6-digit check $2W \cdot 16^{-6}$	7.15e−5

Agreement with prediction is within 9%. The block method is $3.1\times$ worse than an idealised single check, matching the predicted penalty $2^B(1+\gamma)^B/2 = 3.13$. This configuration was chosen because its rate is large enough to measure; the same law extrapolates to the $d = 21$ case below.

5.9 Long sequences: 21 digits via three disjoint blocks

n	worst block loss	found at	corrupted variant
42	3.44e−9	42	correctly rejected
500	3.46e−9	500	correctly rejected
1000	2.93e−9	1000	correctly rejected

Each corrupted variant alters only the final hex digit of 21, and each is rejected. Predicted false-positive rate at $W = 600$: 2.5×10^{-22} . This is the configuration that should be used in place of v1’s unreachable 20- and 40-digit single-evaluation claims.

5.10 Strided sweep

n	stride	evaluations (of 600)	naive	strided	speedup
100	4	26	0.01 s	0.003 s	$3.7\times$
250	4	63	0.04 s	0.011 s	$3.9\times$
500	4	126	0.20 s	0.051 s	$3.9\times$

The strided sweep returned the identical position and classification in every case.

5.11 Summary

All 24 tests pass: 11 positive, 5 against e , 5 against noise, and 3 long-sequence block tests with corrupted controls. The two Monte Carlo experiments confirm the $2W\varepsilon$ law and the block product law to within 9%. V1’s single failure is resolved by the threshold correction rather than excused by it.

6. Discussion

6.1 What kind of classifier is this?

It has no model, no training data, no learned parameters and no feature extraction. It classifies by evaluating a number-theoretic formula at candidate positions and testing for congruence. The decision boundary is not learned; it follows from the structure of π as encoded by BBP.

The guarantees are correspondingly different in character from statistical ones. The false-positive rate is derived by counting over a finite space rather than estimated from a sample. But it is worth being precise about what the derivation assumes: that the W values $f(n)$ behave as independent uniform draws on $[0, 1)$. That is an empirical regularity supported by the π -fraction measurements of Sobieszczanski-Sobieski (2014) and by Section 5.7, and it would follow from base-16 normality of π , which is unproven. The guarantee is combinatorial *given* a probabilistic assumption about π 's digits, not unconditional.

6.2 The role of the positional prior

The classifier needs a bounded window. Without one it would have to search all integers. This is a genuine restriction but not an unusual one: a radiologist knows the image is a chest X-ray, a spam filter knows the input is an email. The prior is the domain knowledge that makes the problem tractable, and it enters the error rate explicitly and linearly through W , which is a reasonable way for a prior to behave.

There is a scaling consequence worth stating. A d -digit sequence first occurs in π at position $\sim 16^d$ on average, so a prior tight enough to be useful becomes harder to supply exactly as longer sequences make the classifier more reliable.

6.3 When to use this rather than a stored expansion

Because π 's hexadecimal expansion is published to vastly more digits than this classifier can reach in reasonable time, indexed lookup is the faster option across the entire practically searchable range. The case for the BBP sweep rests on three narrower situations: positions beyond any available expansion; environments where storage rather than compute is the binding constraint; and settings where one wants a self-contained procedure with no external data dependency. V1 presented the memory advantage as a general one; it is real but conditional.

6.4 Relationship to one-way functions

Forward evaluation is efficient and deterministic, reverse admits no analytic shortcut, the loss surface carries no gradient, and verification is a single evaluation. That shape recalls one-way functions in the sense of Diffie and Hellman (1976). We note the resemblance and claim nothing from it: there is no trapdoor, no secret, no hardness assumption, and, as Section 3 discusses, a practical shortcut through stored digit tables. Readers interested in the hardness of BBP-type inversion should treat it as an open and, to our knowledge, unstudied question rather than as anything established here.

6.5 Limitations

- **Hexadecimal only.** BBP operates in base 16. Base-10 sequences require conversion or Plouffe's 2022 decimal extraction formula, which is less efficient.

- **Positional prior required.** The classifier answers “does this sequence occur in π near here,” never “does this sequence occur in π .”
- **Precision ceiling.** Single evaluations resolve $d_{\max}(n) \leq 11$ digits, falling to 8 near $n = 10^9$. Longer sequences need the block method; higher single-shot precision needs double-double arithmetic, as used in standard BBP implementations.
- **Linear cost in n .** Classification near 10^9 is feasible but takes hours even in a compiled implementation, and that is where the precision budget is tightest.
- **Assumption-dependent guarantees.** As noted in 6.1, the error analysis rests on the empirical uniformity of π ’s hex digits.

7. Conclusion

The Bailey–Borwein–Plouffe formula, evaluated across a bounded window of candidate positions and tested for congruence on a modulo-1 circle, functions as a binary classifier for π -membership of hexadecimal sequences. It requires no precomputed digit tables, no multi-precision arithmetic and $O(1)$ memory.

Its error behaviour is fully determined by one quantity, the effective resolution $\rho = 16^{-d} + \delta(n)$, and one design choice, the threshold $\varepsilon > \rho$, with $P(\text{FP}) \approx 2W\varepsilon$. Tying ε to ρ rather than fixing it is the difference between a false-positive rate of 10^{-2} and one of 10^{-5} at seven digits, and between an error rate that ignores sequence length and one that decays exponentially in it. Where double precision runs out, at about eleven digits, partitioning into disjoint blocks restores the exponential decay at a cost of 2^B .

The broader point survives the corrections, in modified form: the asymmetric computational structure of certain formulas can be repurposed as classification machinery with combinatorial rather than statistical guarantees. What the corrections show is that such guarantees are only as good as the arithmetic they are stated in, and that the resolution of the encoding – not the length of the input – is what ultimately sets the error rate.

References

1. Bailey, D. H., Borwein, P. B., & Plouffe, S. (1997). On the Rapid Computation of Various Polylogarithmic Constants. *Mathematics of Computation*, 66(218), 903–913.
2. Bailey, D. H. (2006). The BBP Algorithm for Pi. Technical Report, Lawrence Berkeley National Laboratory.
3. Bailey, D. H. & Crandall, R. (2001). On the Random Character of Fundamental Constant Expansions. *Experimental Mathematics*, 10(2), 175–190.
4. Percival, C. (2000). PiHex: A Distributed Effort to Calculate Pi. <https://www.cecm.sfu.ca/projects/pihex>
5. Plouffe, S. (2022). A Formula for the n th Decimal Digit or Binary of π and π^2 . arXiv:2201.12601.
6. Diffie, W. & Hellman, M. (1976). New Directions in Cryptography. *IEEE Transactions on Information Theory*, 22(6), 644–654.
7. Sobieszczanski-Sobieski, J. (2014). Pi Fractions for Generating Uniformly Distributed Sampling Points in Global Search and Optimization Algorithms. arXiv:1401.3038.
8. Karrels, E. (2017). Computing digits of π with CUDA. <https://www.karrels.org/pi/>

Appendix A: Implementation

A.1 Classifier

```
"""Memory-free positional classifier for pi-membership of hex sequences."""
import math

# ----- forward engine -----

def bbp_sum(j, n, tail=20):
    """Fractional part of the BBP sub-series for parameter j at position n."""
    s = 0.0
    for k in range(n + 1):          # finite sum, modular
        r = 8 * k + j
        s = (s + pow(16, n - k, r) / r) % 1.0
    for k in range(n + 1, n + 1 + tail): # geometric tail
        r = 8 * k + j
        s = (s + 16.0 ** (n - k) / r) % 1.0
    return s

def bbp_fraction(n):
    """frac(16^n * pi), in [0, 1)."""
    return (4.0 * bbp_sum(1, n) - 2.0 * bbp_sum(4, n)
            - bbp_sum(5, n) - bbp_sum(6, n)) % 1.0

def extract_hex_digits(n, count=7):
    frac = bbp_fraction(n)
    out = []
    for _ in range(count):
        frac *= 16.0
        d = int(frac)
        out.append(f"{d:X}")
        frac -= d
    return "".join(out)

# ----- error budget -----

C_ENGINE = 1.0e-15          # delta(n) ~ C*sqrt(n); measured max C = 5.4e-16

def engine_error(n):
    return C_ENGINE * math.sqrt(max(n, 1))

def resolution(d):
    """Truncation floor: a d-digit prefix pins the value only to 16^-d."""
    return 16.0 ** -d

def default_threshold(d, n_hi, margin=1.25):
    """Smallest threshold still guaranteeing zero false negatives."""
```

```

    return margin * (resolution(d) + engine_error(n_hi))

def max_digits(n):
    """Digits resolvable in one float64 evaluation at position n."""
    return int(math.log(1.0 / engine_error(n), 16))

# ----- inverse engine -----

def hex_to_fraction(hex_str):
    return sum(int(c, 16) * 16.0 ** -(i + 1)
               for i, c in enumerate(hex_str))

def circ_dist(a, b):
    diff = abs(a - b)
    return min(diff, 1.0 - diff)

def congruence_loss(n, target):
    return circ_dist(bbp_fraction(n), target)

def classify(hex_seq, search_lo, search_hi, threshold=None):
    """Naive sweep: one BBP evaluation per candidate position."""
    d = len(hex_seq)
    if threshold is None:
        threshold = default_threshold(d, search_hi)
    target = hex_to_fraction(hex_seq)
    best_loss, best_n = 1.0, None
    for n in range(search_lo, search_hi):
        loss = congruence_loss(n, target)
        if loss < best_loss:
            best_loss, best_n = loss, n
        if loss < threshold:
            return True, n, loss
    return False, best_n, best_loss

# ----- strided sweep -----

def max_stride(d, n, alpha=0.1):
    """Bounded by  $16^{(s-1)*\delta(n)} < \alpha * 16^{-d}$ ."""
    budget = alpha * resolution(d) / engine_error(n)
    if budget <= 1.0:
        return 1
    return max(1, int(math.log(budget, 16)) + 1)

def classify_strided(hex_seq, search_lo, search_hi,
                    threshold=None, alpha=0.1):
    """Identical decisions, ~s-fold fewer evaluations."""
    d = len(hex_seq)

```

```

if threshold is None:
    threshold = default_threshold(d, search_hi)
target = hex_to_fraction(hex_seq)
s = max_stride(d, search_hi, alpha)
best_loss, best_n, evals = 1.0, None, 0
n = search_lo
while n < search_hi:
    f = bbp_fraction(n) # one evaluation serves s positions
    evals += 1
    for i in range(min(s, search_hi - n)):
        loss = circ_dist((f * 16.0 ** i) % 1.0, target)
        if loss < best_loss:
            best_loss, best_n = loss, n + i
        if loss < threshold:
            return True, n + i, loss, evals
    n += s
return False, best_n, best_loss, evals

# ----- disjoint blocks, for d > max_digits(n) -----

def classify_blocks(hex_seq, search_lo, search_hi, block=7):
    """
    Partition into DISJOINT blocks and require every block to resonate.
    Disjointness is what makes the per-block events independent.
    """
    d = len(hex_seq)
    blocks = [hex_seq[i:i + block] for i in range(0, d, block)]
    targets = [hex_to_fraction(b) for b in blocks]
    eps = [default_threshold(len(b), search_hi + d) for b in blocks]
    for n in range(search_lo, search_hi):
        losses = []
        for i, t in enumerate(targets):
            loss = congruence_loss(n + i * block, t)
            if loss >= eps[i]:
                break
            losses.append(loss)
        else:
            return True, n, max(losses)
    return False, None, None

def block_fp_rate(W, d, block=7):
    """ $P(\text{FP}) \sim W * 2^B * 16^{-d}$  for B disjoint blocks partitioning d."""
    B = math.ceil(d / block)
    return W * (2.0 ** B) * 16.0 ** -d

```

A.2 Test suite

```
import random
import numpy as np
from bbp_classifier import *

E_HEX = ("B7E151628AED2A6ABF7158809CF4F3C762E7160F38B4DA56"
          "A784D9045190CFEF324E7738926CFBE5F4BF8D8D8C31D763"
          "DA06C80ABB1185EB4F7C7B5757F5958490CFD47D7C19BB42"
          "158D9554F7B46BCED55C4D79FD5F24D6613C31C3839A2DDF")
MARGIN = 50

def run_tests():
    passed = total = 0

    print("=== POSITIVE (pi) ===")
    for pos in [0, 1, 10, 42, 100, 200, 350, 500]:
        seq = extract_hex_digits(pos, 7)
        lo, hi = max(0, pos - MARGIN), pos + MARGIN + 1
        is_pi, n, loss = classify(seq, lo, hi)
        ok = is_pi and n == pos
        total += 1; passed += ok
        print(f" [{ 'PASS' if ok else 'FAIL' }] n={pos} {seq} "
              f"loss={loss:.2e} eps={default_threshold(7, hi):.2e}")

    print("\n=== WIDE SEARCH (W=600) ===")
    for pos in [7, 75, 250]:
        seq = extract_hex_digits(pos, 7)
        is_pi, n, loss = classify(seq, 0, 600)
        ok = is_pi and n == pos
        total += 1; passed += ok
        print(f" [{ 'PASS' if ok else 'FAIL' }] n={pos} {seq} loss={loss:.2e}")

    print("\n=== NEGATIVE (e) ===")
    for start in [0, 10, 30, 50, 80]:
        seq = E_HEX[start:start + 7]
        is_pi, n, loss = classify(seq, 0, 600)
        ok = not is_pi
        total += 1; passed += ok
        print(f" [{ 'PASS' if ok else 'FAIL' }] {seq} best={loss:.4e}")

    print("\n=== NEGATIVE (noise) ===")
    random.seed(2026)
    for _ in range(5):
        seq = "".join(random.choice("0123456789ABCDEF") for _ in range(7))
        is_pi, n, loss = classify(seq, 0, 600)
        ok = not is_pi
        total += 1; passed += ok
```

```

tag = "PASS" if ok else f"FAIL (false pos n={n})"
print(f"  [{tag}] {seq} best={loss:.2e}")

print("\n=== LONG SEQUENCES (d=21, 3 disjoint blocks) ===")
for pos in [42, 500, 1000]:
    seq = "".join(extract_hex_digits(pos + 7 * i, 7) for i in range(3))
    lo, hi = max(0, pos - MARGIN), pos + MARGIN + 1
    ok_p, n, worst = classify_blocks(seq, lo, hi, 7)
    bad = seq[:-1] + ("0" if seq[-1] != "0" else "1")
    ok_n, _, _ = classify_blocks(bad, lo, hi, 7)
    ok = ok_p and n == pos and not ok_n
    total += 1; passed += ok
    print(f"  [{'PASS' if ok else 'FAIL'}] n={pos} worst={worst:.2e} "
          f"corrupted={'rejected' if not ok_n else 'FLAGGED'}")

print("\n=== MONTE CARLO FP RATE ===")
cache = np.array([bbp_fraction(n) for n in range(600)])
rng = np.random.default_rng(7)
for d in [4, 5, 7]:
    eps = default_threshold(d, 600)
    tg = rng.integers(0, 16 ** d, size=200000) / float(16 ** d)
    hits = 0
    for ch in np.array_split(tg, 40):
        dd = np.abs(cache[None, :] - ch[:, None])
        hits += int((np.minimum(dd, 1 - dd).min(axis=1) < eps).sum())
    print(f"  d={d} observed={hits/200000:.3e} "
          f"predicted={2*600*eps:.3e}")

print(f"\n=== {passed}/{total} tests passed ===")

if __name__ == "__main__":
    run_tests()

```

Appendix B: Notation

Symbol	Meaning
n	Hex digit position in the expansion of π
d	Length of input hex sequence, in hex digits
W	Search window size, $n_{hi} - n_{lo}$
$f(n)$	$\text{frac}(16^n \pi)$, the BBP output scalar
t	Target fraction, the input sequence mapped into $[0, 1)$
τ	Truncation residual, $\tau \in [0, 16^{-d})$
$\delta(n)$	Numerical error of the float64 BBP engine, $\approx 10^{-15} \sqrt{n}$
$\rho(d, n)$	Effective resolution, $16^{-d} + \delta(n)$
$L(n, t)$	Congruence loss, circular distance between $f(n)$ and t
ε	Classification threshold; must exceed ρ
$d_{\max}(n)$	Digits resolvable in one evaluation, $\lfloor \log_{16}(1/\delta(n)) \rfloor$
s	Stride of the strided sweep
B	Number of disjoint blocks

Appendix C: Errata from version 1

C.1 Threshold not tied to resolution (Sections 2.4–2.6). V1 used a fixed $\varepsilon = 10^{-5}$ while claiming $P(\text{FP}) \approx W/16^d$. These are inconsistent: $\varepsilon = 10^{-5}$ certifies roughly $\log_{16}(10^5) \approx 4.2$ hex digits of agreement regardless of d . Measured false-positive rate at $d = 7$, $W = 600$: 1.19×10^{-2} against v1’s claimed 2.2×10^{-6} , a factor of 5300. Section 5.7 shows the rate is flat in d under a fixed threshold.

C.2 Residual misattributed to rounding (Sections 5.1, 5.5 of v1). The $\sim 2 \times 10^{-9}$ losses at correct positions are truncation residuals, verified to printed precision in Section 5.2, not float64 rounding, which is $\sim 10^{-15}$. The corollary claim that losses grow with n from accumulated rounding is contradicted by v1’s own table.

C.3 Arithmetic error in the false-positive table (Section 2.5 of v1). Rows for $d = 10, 20, 40$ report $1/16^d$ rather than $W/16^d$, omitting a factor of $W = 1000$. Correct values are 9.1×10^{-10} , 8.3×10^{-22} and 6.8×10^{-46} . Those figures are in any case unreachable in double precision for $d \geq 20$.

C.4 Overlapping windows do not multiply (Section 2.6 of v1). The stated example (digits 0–7 at n , digits 5–12 at $n+5$) shares three digits, so it yields 13 digits of discrimination rather than 16. Blocks must be disjoint for the product law. The technique was also never implemented in v1.

C.5 “Inverse application” (title, Sections 1, 3). The procedure evaluates BBP forward at every candidate position. No inversion occurs, and no continuous optimisation is possible, as v1’s own loss-landscape measurement shows.

C.6 One-way function claim (Section 3, 6.3 of v1). Reduced to a remark. The chaotic landscape restates the digit-uniformity assumption rather than adding evidence; there is no trap-door or secret; and indexed lookup in published expansions is a practical shortcut across the whole reachable range, contradicting “no known shortcut beyond exhaustive search.”

C.7 Statistical defence of the v1 false positive (Section 5.4 of v1). One collision in 21 trials at $p = 2.2 \times 10^{-6}$ is a 1-in-20,000 event, not something “not unexpected.” It was evidence against the model.

C.8 Timing table (Section 4 of v1). Pessimistic by roughly $10\times$ at $n \leq 10^4$ and optimistic at $n = 10^9$, where a compiled implementation needs hours rather than minutes. Remeasured in Section 4.

C.9 Unreachable precision claims (Sections 2.5, 2.6 of v1). V1 simultaneously claimed a 13-digit float64 ceiling and tabulated rates for 20- and 40-digit sequences. Measured $d_{\max}$ is 11 at small n , falling to 8 near 10^9 .

C.10 Prior art. The scalar $\text{frac}(16^n \pi)$ and its empirical uniformity on $[0, 1)$ were previously studied as “ π fractions” (Sobieszczanski-Sobieski, 2014), which is the natural citation for the uniformity assumption underlying Section 2.5.

Minor. V1’s `bbp_sum` contains an unreachable `if r == 0: continue` guard, since $r = 8k + j \geq 1$ for all $k \geq 0$ and $j \in \{1, 4, 5, 6\}$; and the sign correction `f if f >= 0 else f + 1.0` in `bbp_fraction` is unreachable because Python’s `float %` returns a non-negative result for a positive modulus. Both removed.